1.Start-up concept. Develop a fictional start-up around a problem or opportunity that interests you.

Name: GraphCode AI

Problem

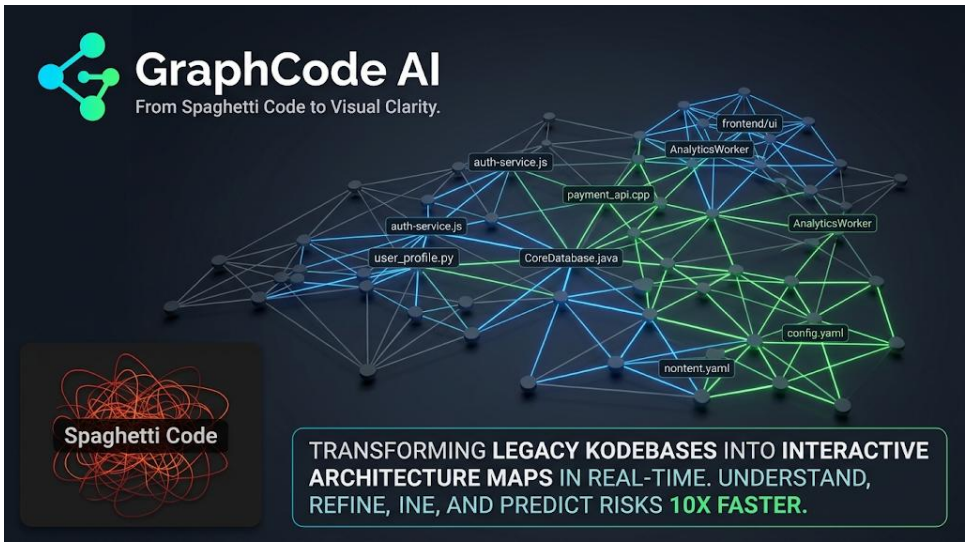
The rapid rise of AI driven code generation has led to an unprecedented level of technical debt. Modern codebases quickly become complex and difficult to navigate, resulting in new developers spending up to 40% of their time simply onboarding and attempting to understand the system architecture.

Solution

A platform that connects directly to Git repositories, combining static analysis with local LLMs to generate a live, interactive visualization of the entire system architecture. This provides developers, especially those exploring the codebase for the first time, with an intuitive, instant overview of how the system functions and how components interact.

Technologies Used

- Code Property Graphs to control flows, and data flows into a unified graph structure.
- Local LLM analysis to ensure zero data leakage by keeping source code analysis entirely within the local infrastructure.
- WebGL and Three.js to create a frontend interface.



2.Simulate a client meeting. **Imagine that your consultancy has been approached by a new client with a challenging product-development problem.**

Client & Context

Client: Marcus Lindqvist, VP of Product at Nordic Logistics AB.

Product: Dynamic Route & Exception Manager, an automated system for rerouting freight during transit disruptions.

Context: Unexpected delays cost millions in contract penalties. Manual rerouting currently takes dispatchers 2 to 3 hours per incident.

Simulated Meeting Dialogue:

Consultant: What is the primary bottleneck in your current dispatch workflow?

Marcus (Client): Speed and visibility. When rail or weather disruptions hit, dispatchers take hours calling truckers and recalculating routes. By then, contract penalties are triggered. We need automated rerouting suggestions within minutes.

Consultant: How should this interface with your existing software stack?

Marcus (Client): It must pull GPS streams from our Telematics API and sync with our core SAP ERP. However, our IT team will not allow heavy modifications to the core SAP database.

Consultant: Understood. Should decision making be fully automated?

Marcus (Client): Not entirely. We want automatic approval for low cost reroutes under 50000kr, but anything higher requires human sign off.

3.Challenge an idea. Choose an idea relevant to your field, for example: software architecture, hardware optimization, sustainable design, business model, manufacturing strategy.

Idea: I want to build my own Operating System. I can do it because Terry Davis built an OS by himself with no help. This will not be difficult.

Benefits, Weaknesses & Trade-Offs

Learning & Technical Mastery:

Benefit: Unmatched mastery of low-level systems programming, CPU registers, memory paging, interrupt handling, and assembly.

Weakness: Massive risk of burnout and project abandonment, as debugging bare-metal kernel crashes without standard tools is notoriously tedious.

Architecture & Control:

Benefit: Absolute total design freedom—zero external dependencies, bloat, or third-party framework overhead.

Weakness: Complete lack of a software ecosystem. Every basic utility—such as a text editor, shell, file viewer, or compiler—must be written completely from scratch.

System Execution & Speed:

Benefit: Tailored execution with instant boot times and lightweight memory footprint.

Weakness: High vulnerability to memory bugs, system lockups, and security exploits unless memory safety and page protection are designed correctly from day one.